Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !’ DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !’ MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !’ FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !’ QA-VALIDACION (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: Página Web Productora 2 (SCRUM-20)

1. CONTEXTO
Se requiere una página web de presentación corporativa para una productora audiovisual. El stack está explícitamente restringido a HTML5, CSS y JS puro. No existe backend propio (el formulario de contacto enviará datos vía servicio externo como EmailJS o Formspree). El entorno de ejecución es LOCAL (protocolo file:///) durante desarrollo, con despliegue final en hosting estático. El requerimiento es un sitio de 4 secciones: Home, Servicios, Quiénes Somos y Contacto.

2. DECISIÓN ARQUITECTÓNICA
Arquitectura elegida: Single-File Application (SFA) con separación mínima de assets.

Se implementará como una Single Page Application (SPA) de scroll suave con navegación por anclas (#hash), contenida en un único `index.html` con un `style.css` y un `main.js` planos (sin módulos ES6, sin import/export). Toda la lógica JS se empaqueta en `main.js` como un único archivo plano compatible con `file:///`.

EVALUACIÓN DE ENTORNO CRÍTICA: El requerimiento especifica entrega en HTML5/CSS/JS solamente, sin servidor de desarrollo garantizado. Por tanto, se aplica la restricción de entorno LOCAL:
- 'L PROHIBIDO: `<script type='module'>`, `import/export`, múltiples archivos JS interdependientes
- ' PERMITIDO: Un único `main.js` con todo el JavaScript concatenado y plano
- ' PERMITIDO: Un único `style.css` con todas las reglas

3. JUSTIFICACIÓN BASADA EN YAGNI/KISS
- **¿Por qué NO microservicios o SPA con framework?** El requerimiento prohíbe explícitamente frameworks. No hay estado complejo que gestionar. YAGNI.
- **¿Por qué NO múltiples archivos JS?** El entorno de ejecución puede ser file:///. Los módulos ES6 nativos fallan sin servidor HTTP. KISS.
- **¿Por qué NO backend propio?** El requerimiento excluye CRM e integración de sistemas. El formulario se maneja vía EmailJS (CDN, sin servidor). YAGNI.
- **¿Por qué NO base de datos?** El requerimiento explícitamente indica que los datos del formulario NO deben almacenarse. YAGNI.
- **¿Por qué una sola página con anclas en lugar de 4 archivos HTML?** Reduce complejidad de navegación, elimina recarga de página, mejora percepción de rendimiento en móvil. Un solo archivo es más fácil de desplegar en cualquier hosting estático.
- **¿Por qué NO Webpack/Vite/bundler?** Stack restringido a HTML/CSS/JS. Sin herramientas de build. KISS.

4. DECISIONES DE STACK (Choose Boring Technology)
- **Imágenes:** URLs de placehold.co y picsum.photos (obligatorio por restricción del requerimiento)
- **Formulario:** EmailJS vía CDN (https://cdn.jsdelivr.net/npm/@emailjs/browser@4/dist/email.min.js) — alternativa: Formspree con fetch(). Sin backend propio.
- **Fuentes:** Google Fonts vía CDN (Inter + Playfair Display para estética de productora)
- **Iconos:** SVGs inline generados por código o Font Awesome vía CDN
- **Animaciones:** CSS puro + Intersection Observer API nativa (sin librerías)
- **Responsive:** CSS Grid + Flexbox nativos, sin frameworks CSS (Bootstrap excluido por restricción de stack)

5. RIESGOS ASUMIDOS POR SIMPLIFICAR
- **Riesgo 1:** EmailJS tiene límite de 200 emails/mes en plan gratuito. Mitigación: documentado en README, cliente debe contratar plan de pago si el volumen aumenta.
- **Riesgo 2:** Sin backend, la protección anti-spam del formulario es limitada (solo validación client-side + honeypot field). Mitigación: honeypot implementado en HTML.
- **Riesgo 3:** CDN de EmailJS requiere conexión a internet. En file:// sin internet, el formulario no funcionará. Mitigación: aceptable para producción en hosting.
- **Riesgo 4:** Sin CMS real (el requerimiento lo menciona pero el stack lo excluye). Mitigación: el contenido está en HTML semántico, editable directamente por personal técnico básico.

6. CONTRATOS DE INTERFAZ
- Variable `EMAILJS_PUBLIC_KEY` en main.js: reemplazar con clave real de EmailJS
- Variable `EMAILJS_SERVICE_ID` en main.js: ID del servicio de email configurado
- Variable `EMAILJS_TEMPLATE_ID` en main.js: ID del template de EmailJS
- Anclas de navegación: `#home`, `#servicios`, `#quienes-somos`, `#contacto`
- Breakpoints CSS: Mobile < 768px, Tablet 768px-1024px, Desktop > 1024px

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO (post-integration-fix) | Iteraciones: 1
- main.js | Estado: APROBADO (post-integration-fix) | Iteraciones: 1
- style.css | Estado: APROBADO | Iteraciones: 2
- README.md | Estado: SKIPPED (doc) | Iteraciones: 0

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 225,760
Tokens de salida (output): 57,544
Total tokens: 283,304
Modelo utilizado: claude-sonnet-4-6

Ø=Üh Ø=Ü» ~24.8 horas-hombre ahorradas
"H squad de 4 personas × 3.1 días

Desglose por Agente:

DISEÑO-ARQUITECTURA: !“4,380 input | !‘1,556 output | 1 llamadas | ~1.9h ahorradas
MICROPLANIFICACION-DISEÑO: !“16,848 input | !‘8,700 output | 4 llamadas | ~5.8h ahorradas
FRONTEND-DESARROLLO: !“77,655 input | !‘38,188 output | 7 llamadas | ~8.3h ahorradas
QA-VALIDACION: !“126,877 input | !‘9,100 output | 9 llamadas | ~8.7h ahorradas